



Algoritma & Struktur Data

Week 11 - Tree



Last Week Review

- Stack
 - Last In First Out
 - Representing the plate that piling up into a stack
 - Got several method:
 - Top
 - Pop
 - Push
 - Implementation of Stack:
 - Palindrome
- Queue
 - First in First out
 - Representing the LINE of queue like Cinema Ticketing
 - Got several method:
 - Enqueue
 - Dequeue
 - Front
 - Implementation of Queues:
 - Resource Monitor

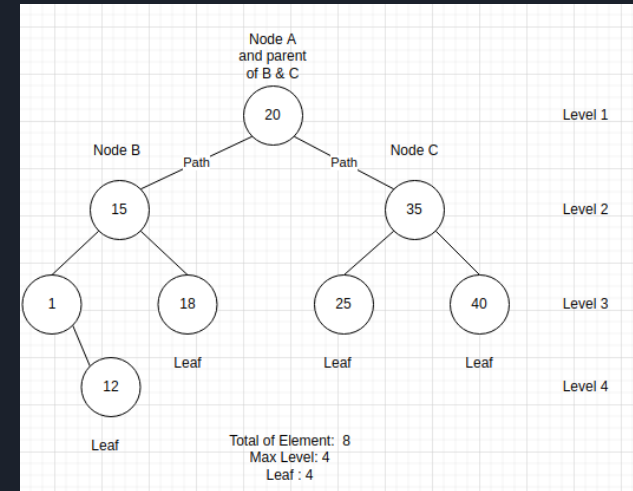


Outline

- Tree

Tree (Anatomy)

- Non Linear Data structure
- Used to Represent data with a hierarchical Relationship.
 - Records
 - Family Trees
 - Menu & Sub Menu
 - Decision Making System
 - Directory Structure of a file manager
 - Arithmetic expressions data structure
 - Can be used to create compression algorithms ex: LZW compression
- Several Terminology for Tree
 - Node: The Element
 - Root: Starting Node
 - Parent: Predecessor element
 - Child: Successor element of the Predecessor
 - Path: Connecting Line
 - Leaf: Childless Node
- Rule of Tree
 - There is only one Root
 - Depends on the terms. One Node may have multiple Nodes



Tree Traversing Method

- Preorder

```
void preorder(Node *root){  
    if(root == NULL) return ;  
    printf("%c",root->data);  
    preorder(root->left);  
    preorder(root->right);  
}
```

Expected output: 20, 15, 1, 12, 18, 35, 25, 40

- Inorder

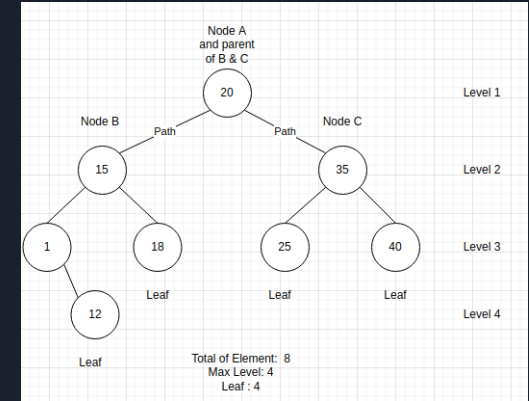
```
void preorder(Node *root){  
    if(root == NULL) return ;  
    inorder(root->left);  
    printf("%c",root->data);  
    inorder(root->right);  
}
```

Expected output: 1, 12, 15, 18, 20, 25, 35, 40

- Postorder

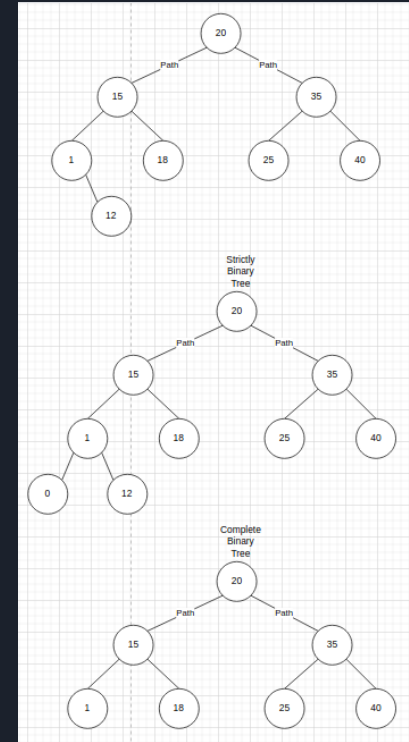
```
void postorder(Node *root){  
    if(root == NULL) return ;  
    postorder(root->left);  
    postorder(root->right);  
    printf("%c",root->data);  
}
```

Expected output: 12, 1, 18, 15, 25, 40, 35, 20



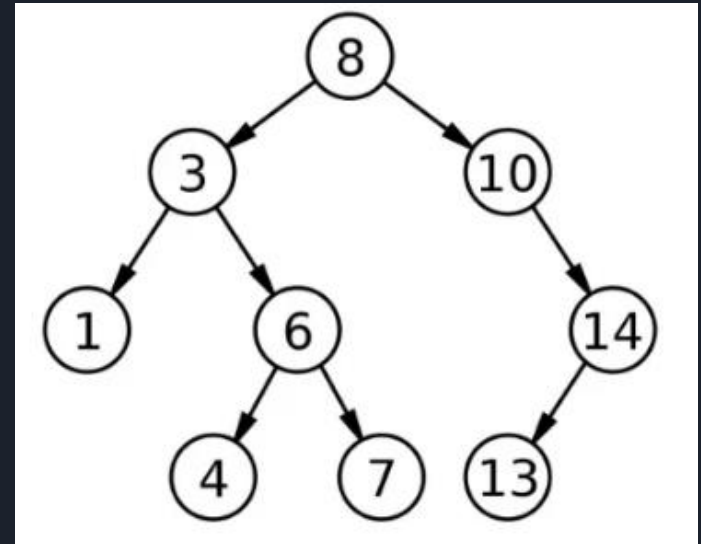
Binary Tree

- Tree where each individual node will not be able to have more than 2 child element
- There is three Variation for Binary Tree
 - Complete Binary Tree (where each child except leaf have two child). All leaf needs to be in the same level
 - Strictly Binary Tree (Where each node except leaf have 2 child) with no regards of the leaf level uniformity.
 - Incomplete Binary Tree (Binary tree with at least one node have only one child) and/or all leaf is not on the same level.



Binary Search Tree

- Binary Tree where all left sub-tree elements are smaller than the root
- Binary Tree where all right sub-tree elements are smaller than the root
- All sub-tree is also a binary tree.





Binary Tree Insertion

```
void insert(Node *root, int value){
    if(root == NULL){
        root = (Node *) malloc(sizeof(Node));
        root->left = root->right = NULL;
        root->item = value
    }
    else{
        if(value < root->item){
            root->left=insert(root->left,value);
        }
        else{
            root->right=insert(root->right,value);
        }
    }
}
```




Binary Tree Searching

```
void search(Node *root, int value){  
    if(root == NULL){  
        printf("data is not exist");  
    }  
    else if(value == root->item){  
        print("data is exist")  
    }  
    else if(value < root->item){  
        search(root->left,value);  
    }  
    else{  
        search(root->right,value);  
    }  
}
```



THE END